一、任务与数据

1.1 任务定义

赛题要求在一个**标签含噪声**的细粒度图像数据集上，基于 CLIP ViT-B/32 完成一次 鲁棒微调，最终对测试集输出 Top-1 预测。评测只看 Top-1 Accuracy 这一个数字。

这意味着两件事。第一，任何方法的取舍都得回到"它能不能提高正确率"上，过程中那些 好看的指标（比如在含噪标签上算出来的验证准确率）都不算数。第二，由于训练标签本身 有错，模型"学得太好"反而可能是坏事——把错误标签也记住，测试时的泛化能力会掉。

1.2 数据规模与噪声特性

先把数据的底摸清。扫描阶段（`pipeline.py scan`）的实测结果：

项	数值
训练集规模	148,695 张
测试集规模	37,444 张
类别数	750
类别编号	`0000` – `0749`，0 起连续，映射无歧义
噪声率（混淆矩阵口径）	约 38.0%
转移矩阵对角线均值	0.6199

噪声率这个数字需要打个折扣看。 我们采用的估计口径是贝叶斯反演 `T[j,k] = A[k,j]·π_obs[k] / π_true[j]`，它会把"模型自身的分类误差"也算进噪声里， 因此系统性偏高。合理的判断是这个 38% 是个上界，真实值大概率低于它。 这个问题在 4.2 节展开。

长尾程度也做了体检（`scripts/scan_longtail.py --real`），结论是**比预想的轻得多**：

指标	实测
头尾类样本数之比	49.6×
Gini 系数	0.0956
有效类别数	736.1 / 750（98.1%）
训练先验熵比	0.9972

训练先验的熵比 0.9972 意味着类别分布**近乎均匀**。这一点在后面对"先验修正" 这一步的判断上起了决定性作用（见 5.4 节）。

1.3 提交与校验

提交文件为 `pred_results.csv`，每行两个字段：图片文件名与四位类别编号，压缩为 zip。 我们写了一个独立的格式核验脚本 `scripts/check_submission.py`，在每次生成提交后必跑。 它的检查项包括：zip 内

条目名、行数、编号是否四位数字、文件名是否与测试集完全一致、 有无重复文件名、预测覆盖的类别数。

这个脚本不是形式主义。本项目在开发过程中出现过一次"官方清单匹配率不足但只打印警告就继续" 的情况——产物**格式完全合法**，只是少了一部分行，任何格式检查都查不出来， 只能等排行榜给一个莫名其妙低分。现在的实现是匹配率低于 99% 直接抛错。

二、方案总体设计

2.1 路线选型

赛题只限制"必须用 CLIP ViT-B/32 与官方权重"，并没有限制怎么用它。摆在面前的有三条路：

方案	单轮成本（实测）	赛程内能跑多少组对照
全参数微调视觉塔	数小时 / 轮	个位数，基本不可行
LoRA 解冻骨干（端到端，含反向）	5.1 分钟 / 轮	一天十几组
冻结骨干 + 缓存特征 + 线性头	提特征一次 10.5 分钟，此后**每轮 10–22 秒**	**一天几十组**

开发机是一台 RTX 4050 Laptop（6 GB 显存）。**真正的约束不是单轮能不能跑完， 而是总共能试多少种配置。** 抗噪机制的对比天然要横向铺开十几组 （损失函数 × 加权方式 × 系数），只有把 CLIP 特征一次性提取并缓存下来， 单组配置的代价才降到秒级，调参和消融才做得动。

所以主链路选冻结路线：特征提一次，之后全部实验都在 512 维特征上做小矩阵乘法。 端到端路线没有废弃， `src/lora.py` 与 `src/finetune_lora.py` 保留下来， 专门用于检验一个关于模型容量的假设（见第六章）。

2.2 流水线

```
scan → extract --rebuild → oof → train → predict → check_submission
```

阶段	做什么
`scan`	建立文件清单、推断并审计类别映射、统计训练先验
`extract`	用冻结的 CLIP 提取 512 维特征并缓存
`oof`	K 折交叉验证产生 OOF 概率，据此估计噪声转移矩阵 T
`train`	在缓存特征上训练鲁棒分类头
`predict`	推理测试集，写出提交文件
`check_submission`	提交格式机器核验

有一个依赖关系在文档里不容易看出来，值得单独提一句：**换阶段或从零复现时， `extract` 必须带 `--rebuild`**。这个参数会先清空上一轮的派生缓存。 我们最初的实现只删特征文件，漏掉了 4 个同样由当前

训练集派生的文件， 其中 `T.npy` **没有任何 shape 自检**——如果不清，它会被静默复用， 等于拿另一套数据的噪声结构去训练本阶段的模型。详见 6.1 节。

2.3 把算力花在方法迭代上

这条路线选择带来一个额外好处：所有实验产物（特征、OOF 概率、先验）都是一次性算好、 可复用的。这让我们在正式训练之前就能做完以下全部工作，而不必反复前向：

- 长尾体检与类平衡权重标定；
- 8 组损失/加权配置的对照；
- 类平衡系数 β 的扫描；
- 先验修正对预测的实际影响诊断；
- 冻结路线与 LoRA 路线的受控对照。

这些实验的结论构成了本方案里几乎所有的设计依据。没有它们，第五章那些取舍就只能靠猜。

三、抗噪方法

分类头本身很简单：一个 512×750 的线性层，外加一个可学习的 logit scale。 复杂的地方全在**损失函数与样本权重**上。我们实现了四个可独立开关的组件。

3.1 冻结特征

CLIP ViT-B/32 在 224×224 输入下输出 512 维特征。骨干参数完全不动， 只有分类头参与训练。这样做的代价是丢掉了细粒度任务上可能存在的微调收益， 但换来的是可控性与可复现性——特征一旦缓存，后续任何实验结果都能被完整复算。

预处理是自己实现的一版 torchvision 管线，没有直接用官方的 `CLIPImageProcessor`。 这不是随手写的：我们用 480 张图对比验证过两者的数值等价性（4.1 节）， 自写版单进程快约 1.6 倍，特征空间余弦相似度最低 0.999888。 如果需要与官方处理器逐位一致的复核口径，设 `AIC_PREPROC=official` 即可切换。

3.2 鲁棒损失

框架里实现了 `ce` / `sce` / `gce` / `forward_ce` / `forward_sce` / `forward_gce` 六种损失， 由 `--kind` 切换。这里只展开实际用到和实际被淘汰的几种。

对称交叉熵（SCE）： `L = α·CE + β·RCE`，其中 `RCE = -Σ_j p_j·log p_j`。 RCE 这一项**完全不含标签**，它的作用是惩罚过于尖锐的预测分布； 在 K 类上它的极小值恰好是均匀分布。

这里有一个我们必须记下来的教训，因为它跟端到端微调直接相关：

> **默认的权重配比 α=0.1 / β=1.0 在端到端场景下会把准确率摁死在 1/K。** 实测（20 类 / 800 张 / LoRA r=16 / 8 轮，其余参数完全一致，单变量）： β=1.0 时 val_acc **0.0375**，而 20 类的随机水平就是 0.05 —— 而且 loss 从 1.61 一路降到 1.08，**看起来完全像是在正常收敛**。把 β 降到 0.05 后 val_acc 是 0.5375， 朴素 CE 是 0.5312。问题不在 LoRA，在于 RCE 的权重是 CE 的十倍。

> 为什么在只训线性头时没暴露？因为那种情况下输入是**已经含有类别判别信息的冻结特征**， 线性头只要放大特征方向就能压过 RCE 的均摊压力。端到端时骨干附近的梯度更弱，RCE 就赢了。 这是端到端路线特有的失败模式。

因此配置里 `SCE_BETA` 已从 1.0 改为 **0.05**，并在训练出口加了护栏： 若最优准确率不超过 `1.35/K`，直接报警并打印"改用纯 CE"的命令， 同时明确写清"**不要**把这个现象当成 LoRA 无用的证据"。

前向校正： `L = -Σ_j T[y,j]·log p_j` ，用转移矩阵把损失前向到观测标签空间。 它需要 T，而 T 的估计是有偏的（4.2 节）。**本次提交未启用这个组件**， 具体理由见 3.5 节。

3.3 CL 保真度降权

用 OOF 概率与含噪标签估计出 750×750 的转移矩阵 T，取对角线作为逐类保真度， 按 `w_i = clip(T[y_i, y_i], 0.30, 1.00)` 给每个样本加权：标签越不可信的类， 其样本在损失里被压得越低。

实测的权重分布是这样的：11,229 张（7.6%）被压到地板 0.30， 只有 178 张（0.1%）拿到满权重 1.0，中位数 0.6347。 换句话说，**这个机制对几乎所有样本都做了惩罚**，只是程度不同。

这个分布一开始让我们担心：如果大部分样本都被降权，会不会反而拖累模型？ 5.2 节的对照实验回答了这个问题——开启与关闭 CL 降权，覆盖度指标完全相同。

3.4 长尾类平衡加权

`w_c = (1-β)/(1-β^{n_c})`，源自 Cui 等人的类平衡损失。 它给样本数少的类更大的权重，缓解长尾下模型偏向头类的问题。

β 的取值见 5.3 节的扫描结果，最终用 **0.99**。这里另有一个必须点名的地雷：

> **β = 1.0 会让全部样本权重变成 NaN，而且训练不会报错。** 分子 `1-β` 变成 0，于是 `w_c` 全为 0，归一化那一步 `w_c / w_c.mean()` 就是 `0/0` 。 实测 148,695 个权重全是 NaN，loss 静默变成 nan，val_acc 掉到 0.0016， **但训练照常跑完 60 轮并正常保存了模型。** 原有的 `np.maximum(eff, 1e-12)` 只护住了分母，护不住分子的 0。 现在在两处拦截：函数入口校验 `0 < β < 1`，以及训练循环里在 `backward()` 之前检查 `torch.isfinite(loss)` 。

这个坑的一般意义值得写下来：**凡"取极端值就该退化到某个已知简单行为"的参数， 都要把极端值本身跑一遍。** β→1 的数学极限是"权重全相等"， 数值实现里却是 `0/0` 。数学极限与数值实现不一致的地方，就是地雷所在。

3.5 组件的实际启用情况

方法框架里有四个组件，但**提交时实际生效的只有其中三个**。这一点必须写清， 因为把没跑起来的模块算进方法贡献，是一种很难被发现、但性质上属于虚报的错误。

组件	提交时状态	说明
鲁棒损失	✅ `--kind sce` （α=0.1，β=0.05）	5.2 节显示 SCE 系显著优于朴素 CE
CL 保真度降权	✅ 开启， `CLIP_LO = 0.30`	5.2 节实测覆盖度无损失

组件	提交时状态	说明
长尾类平衡	✅ `--cb cb`，β = 0.99	5.3 节扫描确定
前向校正	❌ **未启用**	依赖的 T 两个估计方向都有偏，见 4.2 节
测试先验修正	❌ `--prior none`	先验近乎均匀，修正基本空转，见 5.4 节

四、方法自检

在真实数据上的对照之前，先用合成特征把装置本身验一遍。这一步的目的是回答 "机制到底有没有在起作用"，而不是"哪个配置更好"——后者必须靠真实数据。

4.1 合成特征上的配对比较

任务设定：20 类 / 训练 4000 / 干净测试 2000 / 注入 30% 标签噪声 / 噪声范数与类中心范数之比 1.0。用 **6 个随机种子做配对比较**（同种子作差，公共方差相互抵消）：

配置	准确率	Δ 相对朴素 CE	配对 t
朴素交叉熵（下限）	0.8704 ± 0.0207	—	—
SCE	0.9285 ± 0.0427	+0.0581	+3.09
SCE + 类平衡	0.9163 ± 0.0421	+0.0458	+2.67
SCE + 类平衡 + CL 降权	**0.9343 ± 0.0262**	**+0.0638**	**+6.01**
完整方案（前向校正 s=0.5）	0.9369 ± 0.0217	+0.0665	+5.52

两个结论：SCE 系全部显著优于朴素 CE（+4.6 到 +6.7 个百分点，配对 t 均大于 2.5）； SCE 系内部前三名的差距**统计上不可区分**，谁更好必须回到真实数据上比。

> 这里有过一次被推翻的结论。早期只用 3 个种子时，我们观察到"前向校正 s=0.5 掉了 3 个百分点"，据此把默认配置改成了不带前向校正的 SCE。加到 6 个种子并改用 配对比较后，前向校正反而均值最高（0.9369），之前的差异只是噪声。 **变体之间只差几个百分点时，3 个种子根本不够。** 这个教训后来在别处又用上了一次。

顺带说明预处理等价性（3.1 节提到的那 480 张图）： 自写管线与官方处理器在 归一化空间的最大绝对差是 3.002e-02，换算到像素灰阶峰值 **2.000/255**、 均值 **0.0090/255**。差异只来自双三次核的实现细节，属舍入级； 在特征空间里两者余弦相似度最低 0.999888、均值 0.999981。

4.2 转移矩阵估计：两个方向都有偏

同一套自检里测了 T 的估计精度（真实噪声率 0.30）：

估计器	估计值	误差
`confident_joint`（原始实现）	0.0014	0.2986

估计器	估计值	误差
`confusion`（贝叶斯反演）	0.4645	0.1645

两个方向都不对，原因不同。

`confident_joint` 假设模型会"记住"噪声标签，而我们的路线是 "强特征 + 线性探针"的**泛化**模式：被错标的样本会被 OOF 高置信地判成真实类， 恰好被它的阈值筛掉，于是 T 退化成单位阵。

`confusion` 口径则反向高估，因为它把模型自身的分类误差也算进了噪声。

这是本方法的固有难点，我们没有解决它，而是绕开了它。 具体做法是： 默认损失用 SCE，它的正则项不含标签、因而不依赖 T；前向校正虽然实现了， 但在 T 不可靠的前提下不启用。

五、真实数据上的配置对照

5.1 诊断量的选择

真实测试集没有标签，没法直接算准确率。**而在含噪训练标签上算出的准确率是不可用的**： 抗噪方法"故意不拟合噪声"，在含噪指标上反而显得更差。这一条有实测支撑—— 早期四个配置在含噪验证集上的准确率差不到 3 个百分点（0.6550 vs 0.6800）， 而真实榜分差 2.24 分，指标完全没有区分力。

所以我们改用一组**不依赖标签**的诊断量：训练集拟合率、预测覆盖类别数与覆盖率、 预测熵比、最大类占比。它们不回答"哪个分高"，只回答"模型有没有塌缩"。

这里也踩过一个坑，记录如下：

> 对照表的第一版里有一列"代理准确率"，读的是 OOF 阶段预先算好的**全局**概率矩阵。 那个矩阵独立于当前配置，于是 8 个配置算出了**完全相同的数** （0.6233094589596153，小数点后 16 位一致）。危险之处在于它长得完全正常——0.6233 是个合理准确率，不逐列对比根本发现不了， 据此选配置等于瞎选。 现在做了两件事：改用本配置自己的模型计算；并加了一条**常数列护栏**， 跑完自动扫描所有指标列，凡是在所有配置上取值完全相同的，直接点名拒绝解读。

5.2 八组配置的结果

配置	损失	CL降权	类平衡	T收缩	训练集拟合率	覆盖类别数	覆盖率	预测熵比	用时s
朴素 CE（下限）	ce	否	none	—	0.8042	743	0.9907	0.9612	12.7
GCE	gce	否	none	—	0.6954	**606**	**0.8080**	0.9411	9.9
SCE	sce	否	none	—	0.9167	746	0.9947	0.9713	10.2
SCE + 类平衡	sce	否	cb	—	0.9161	746	0.9947	0.9720	11.2
SCE + 类平衡 + CL降权	sce	是	cb	—	0.9118	746	0.9947	0.9723	10.8

配置	损失	CL降权	类平衡	T收缩	训练集拟合率	覆盖类别数	覆盖率	预测熵比	用时s
前向校正 CE (s=0.5)	forward_ce	否	none	0.5	0.8748	745	0.9933	0.9764	10.1
完整方案 (s=0.5)	forward_sce	是	cb	0.5	0.8560	**748**	**0.9973**	0.9774	12.9
完整方案 (s=1.0)	forward_sce	是	cb	1.0	0.7824	727	0.9693	0.9684	15.0

判读：

GCE 应该淘汰。 它的覆盖率只有 0.808（606/750 类），是 8 组里唯一明确塌缩的配置。 GCE 的 `q=0.7` 在这个噪声率下把梯度压得太狠，模型退化成只覆盖大部分类别。 其余 7 组覆盖率在 0.969 到 0.997 之间。

CL 保真度降权是无害的。 `SCE+类平衡` 与 `SCE+类平衡+CL降权` 的覆盖类别数完全相同 （都是 746/750），拟合率只差 0.004。考虑到 3.3 节那个"77% 样本被降权"的分布， 这个结果说明降权带来的负面影响被抗噪收益抵消掉了。

拟合率和覆盖率要一起看。 拟合率低不一定是坏事——不拟合噪声本来就是目的； 但**覆盖率低一定是坏事**，那是塌缩。所以最终用覆盖率做淘汰判据。

5.3 类平衡系数 β

固定"完整方案 s=0.5"这一档，扫描 β：

β	类平衡跨度	组合跨度(×CL)	有效样本比%	覆盖类别数	覆盖率	训练集拟合率
0.9	2.4×	8.1×	91.8	743	0.9907	0.8568
0.99	**18.7×**	**62.3×**	**89.7**	**748**	**0.9973**	**0.8570**
0.999	44.0×	145.7×	82.1	748	0.9973	0.8560
0.9999	49.0×	162.0×	80.4	748	0.9973	0.8557
1.0	NaN	NaN	—	1	0.0013	—

选 **0.99** 的理由：它与 0.999 的覆盖度完全等价（都是 748 类 / 0.9973）， 但权重跨度小一半，对有效样本数的侵蚀也更小。

原来默认的 0.999 是对着比实际情况严重得多的长尾设的。1.2 节的体检显示， 真实头尾比只有 49.6×，用 0.999 相当于对着一个不存在的困难做防御。

需要说明的是，**覆盖率只能证明"模型没塌"，不等于榜分更高。** 这张表的用途是排除明显有害的取值。

5.4 先验修正：在本数据上基本是空转

"测试先验修正"是个常见技巧，靠 $p(c|x) \propto p(c)\cdot p(x|c)$ 把训练集先验的影响扣掉。 但它的收益依赖一个前提：**测试集与训练集的类别分布不同。** 本例中训练先验熵比 0.9972，近乎均匀，所以先量一量它到底改了多少：

对比	改变的预测数	占比
乘 `1/train_prior`	919 / 37,444	**2.45%**
乘 `1/EM估计`（加下限约束）	5,258 / 37,444	14.04%
乘 `1/EM估计`（**无约束，修复前**）	23,029 / 37,444	**61.50%**

乘 `1/train_prior` 只改动 2.45% 的预测，因为先验近乎均匀时 `p/q` 近似常数， argmax 基本不变。这等于没做，不值得占用提交机会。所以首次提交用 `--prior none` 拿一份干净基线。

而那个"修复前"的 61.50% 是一次严重的事故，记录如下：

> **无约束 EM 在本数据上会反向塌缩。** EM 每轮乘 `q/q_tr` 且完全无约束， 迭代 200 轮后头尾比被放大到 **6.4e49**（最小类 1.9e-52）， 于是 `p / q` 溢出到 5e48 —— **先验越小的类越必然胜出。** 实测第 728 类一度占到 25.8% 的预测，整体改动了 61.5% 的预测。 如果不是出提交前做了一次预演，这个版本会直接交上去。
>
> 修法两层：EM 内部加 `min_ratio=0.1` 的夹子（单类调整幅度不超过 10 倍）； 出提交前预演一次修正，检查**方向**，方向反了直接拒绝产出。
>
> **这里还踩了第二次坑**：护栏的第一版判据写的是"头尾比 > 1e4 就拦"， 结果把**健康**的 EM 也误拦了——它加夹子后头尾比仍有 6.2e05， 但胜出类的先验均值是全体的 5.37 倍，方向是对的。 **头尾比大只说明"估出的先验形状尖"，不等于病态；真正的病态是方向反了。** 教训是：先想清楚病态的机理，再决定量什么指标。阈值卡的形状与机理不等价时， 第一个阈值必然误伤。

六、容量假设的检验

这一章是一次**假设被否证**的记录。写进来是因为它的结论直接改变了方案的主线方向， 而中间那个错误的判读过程也很典型。

6.1 假设的来源

赛道上一阶段的实测榜分是：把一整套抗噪方法加满得 **52.04**， 最朴素的交叉熵得 **49.80**，差 **+2.24**。

一整套方法只值 2.24 分，这是个偏低的幅度。一个自然的解释是： **瓶颈不在损失函数，而在模型容量。** 骨干完全冻结时只有一个 512×K 的线性头可调， 抗噪方法能腾挪的空间被卡死了。如果这个解释成立，那么提分方向就是解冻骨干。

这个解释看起来很有说服力，而且在当时的证据下几乎是唯一合理的解释。 **但它是一个推断，不是测出来的。**

6.2 实验装置

为此写了一个对照实验装置（`scripts/run_ab_harness.py`），刻意规避了三个陷阱：

1. **固定训练步数预算。** 否则线性头能跑几百轮、LoRA 只能跑几轮， 差距里会混进"谁被允许训更久"这个无关变量。
2. **每类留出干净的样本**并从训练中剔除。含噪验证集没有区分力（5.1 节）。

3. **报告预测覆盖类别数。** 训练塌了就必须拒绝给结论， 否则会把"训练塌了"读成"方法更差"。

配置：150 类 / 每类 100 张训练 / 每类 20 张干净留出（n=3000）/ LoRA r=16 / lr=1e-4 / batch=96 / 两边同为 8 轮。

6.3 结果

版本	冻结线性头	LoRA r=16	Δ	判读
160 轮 vs 8 轮（预算**未**对齐）	0.7670	0.7403	−0.0267	显著（z=2.40）
8 轮 vs 8 轮（预算已对齐）	**0.7603**	**0.7403**	**−0.0200**	**不显著**（95% CI 跨 0）

两条路线都覆盖 150/150 类，没有退化，所以装置是健康的，差距不能用"塌了"解释。

6.4 结论与边界

那个"LoRA 落后且显著"的印象，主要来自"冻结线被允许多训 20 倍"。 把预算对齐之后，差距落进噪声范围内。在本实验规模上，解冻骨干没有带来可观测的收益。

但**这不足以否定 LoRA**，有三条边界必须写清：

- 子集只有 150 类（全量 750 类）。任务偏简单，冻结线已经接近饱和 （0.76 这个水平留不出多少提升空间）；
- 为公平起见两边只训 8 轮，LoRA 未必收敛充分；
- LoRA 的学习率未必是这个规模下的最优值。

另外要强调：**留出集准确率不能外推成榜分预测。** 类别少意味着任务简单， 这个 0.76 必然偏高。它唯一的用途是"在同一条件下比较两条路线"。

策略含义：LoRA 的单轮成本是冻结路线的约 300 倍（1774 秒 vs 5.6 秒）， 冻结路线一天能跑几十组对照，LoRA 只能跑 8 到 10 组。 因此提交主链路继续用冻结线性头（已验证、可控、快），LoRA 作为并行的探索方向保留。

> 顺带记一个走不通的路：我们试过用**合成图形**把任务难度放大， 好让容量差异显现出来。这条路不通。CLIP 对 PIL 画的扁平色块， 表征挤在很窄的锥里——实测**纯红满屏与纯蓝满屏**的类心余弦相似度就有 0.913， 32 个色块均值 0.968、最大 0.995。任务的可分性上限由"CLIP 怎么看合成图"决定，**不由骨干容量决定**，冻结与解冻都学不动，两次都塌成"全预测同一类"。 要验证容量这类问题，只能用真实图像。

七、性能标定

全部数据在 RTX 4050 Laptop（6 GB 显存 / 20 逻辑核 / 16 GB 内存）上实测。

7.1 全链路耗时

阶段	数据规模	实测耗时	吞吐
`extract` 训练集	148,695 张	**10.5 分钟**	236.6 img/s

阶段	数据规模	实测耗时	吞吐
`extract` 测试集	37,444 张	**1.2 分钟**	499.4 img/s
`oof`（5 折）	148,695 张	**52 秒**	—
`train`	60 轮	**22 秒**	—
`predict`	37,444 张	数秒	—

全链路约 13 分钟（不含环境安装）。特征提取占了绝大部分， 其余阶段都在缓存的 512 维特征上做矩阵乘法，所以很快。

7.2 worker 数：两种场景的结论正好相反

这是一个容易照搬经验出错的地方。

提特征（只做前向）：512×512、n=6144、batch=96 下， 单进程吞吐 79.8 img/s，其中 **90% 的时间耗在"等数据"**（JPEG 解码 + 缩放 + 归一化）， GPU 只占 9.9%。开 6 个 worker 后吞吐升到 339.8 img/s，**加速 4.26 倍。** 用 `t(n) = F(W) + n/c(W)` 解出固定开销：F 在 worker 数上线性累加（约 2.5 s/进程）， 因为 Windows 下 spawn 子进程要重新 import torch / numpy。

端到端训练（含反向）：真实 JPG 3072 张、batch 96 下实测：

workers	ms/步	吞吐	可用内存
0	**198**	**484.6 img/s**	**3455 MB**
8	—	422.2 img/s	917 MB

worker 越多越慢。 原因是训练要做反向，瓶颈在 GPU，此时 worker 的 spawn 固定开销成了净负担；而提特征只做前向、瓶颈在 CPU 解码，worker 才是收益项。

所以训练侧的默认值改成了 `--workers 0`。**这两类场景的 worker 配置不能互相照搬。**

7.3 一个被高估 5 倍的排期

在做端到端路线的时间预估时，我们用"前向 + 反向约为前向的 2.5–3 倍"这个经验倍数， 算出全量单轮约 25 分钟。实测是 **5.1 分钟**（198 ms/步）。

这个倍数在本场景高估了 5 倍。**排期必须建立在实测上，经验倍数只能用来做数量级判断。**

八、工程可靠性、合规与复现

8.1 阶段隔离

赛题规则明确禁止跨阶段复用数据。这类违规的可怕之处在于**它往往是静默的**： 同名文件的 shape 不报错、程序照跑、结果看起来正常。

我们的实现方式是**两个物理隔离的工程目录**：本阶段工程的数据目录与输出目录 从空开始建立，只复用模型代码与 OpenAI 官方权重。数据集本身不使用任何前序阶段的内容。

除目录隔离外还有两道程序化防线：

清理清单。 换阶段时由当前训练集派生的文件实际有 6 类，最初的实现只删了 2 类：

文件	是否曾漏删	危害
`feat_train.npy` / `feat_test.npy`	已删	—
`oof_probs.npy`	❌ 漏删	体积大，由另一阶段训练集算出
`T.npy`	❌ 漏删	**无任何 shape 自检 → 会被静默复用**
`train_prior.npy`	❌ 漏删	类别先验错配
`train_meta.json`	❌ 漏删	存着另一阶段的 `num_classes`，分类头规模错配

`oof_probs.npy` 侥幸能自保（有行数检查，规模不同会触发重建），**但 `T.npy` 什么都没有，是最危险的一个。**

类别数护栏。 非 rebuild 模式下比对 `train_meta.json` 的 `num_classes` 与当前训练集实际类数，不一致即拒绝运行并提示改跑 `--rebuild`。 实测伪造残留元信息可以被拦住。

还有第三道，是因为前两道都不够。 上面的护栏依赖"能读到数据"来判断阶段， 而清理数据这个动作恰好会把判据一起清掉——**护栏会在最需要它的时候失效。** 所以补了一条不依赖被保护对象的兜底判据（按工程根目录名识别阶段）。

> 一般化的教训：**"守卫依赖被守卫对象"是最隐蔽的护栏缺陷。** 凡是"靠被保护对象判断是否需要保护"的逻辑，都要有一条独立的兜底判据。

8.2 静默错误清单

以下问题都属于"程序照常跑完、输出看起来正常、但结果是错的"，因此都加了程序化拦截。

现象	根因	处理
loss 静默变 NaN，训练正常跑完	类平衡 β=1.0 使分子为 0（3.4 节）	入口区间校验 + `backward()` 前查 `isfinite`
8 个配置的诊断指标完全相同	读了独立于配置的全局中间产物（5.1 节）	改用本配置模型 + 常数列护栏
先验修正反向塌缩，改动 61.5% 预测	无约束 EM 数值发散（5.4 节）	EM 加夹子 + 出提交前方向检查
各配置在含噪验证集上准确率几乎无差异	指标本身对鲁棒方法不敏感	弃用该指标，只作塌缩检测
对照实验结论自相矛盾	`int(os.environ.get(k) or 0) or 默认值` 把显式设的 `0` 覆盖成默认值	改显式判空；并打印"实际生效"的参数值
备选配置的产物覆盖了正式提交件	训练先无条件写主权重再复制副本	改为按 tag 直接落盘，互不覆盖

现象	根因	处理
提交文件残缺但格式合法	官方清单匹配不全时只打印警告就继续	匹配率 < 99% 直接抛错
AMP 下学习率提前退火	`scaler.step()` 跳过 `opt.step()` 时仍调用 `sched.step()`	用 `scaler.update()` 后的 scale 变化判断跳步

这一节的共同教训是：**"跑完了、有数字、格式对"不等于结果可用。** 本项目在开发过程中连续三次都是在看起来成功的输出里发现假信息。

8.3 位级复现

方案文档里写的配置**不能只凭记录**。我们做了一次直接核验：

1. 按最终配置重跑 `train`，得到的分类头权重与提交用权重**逐位完全相同**（`np.array_equal` 为真，最大绝对差 **0.0**）；
2. 分别用三种先验设置重跑 `predict`，只有 `--prior none` 的结果与提交文件 **逐字节相同**（另两者分别有 919 行、5258 行差异）；
3. 模型文件内部记录的 `kind` 字段与文档声明一致。

这三次核验的成本不到一分钟，但它排除了一类很麻烦的错误—— **同一份配置写在多处文档里，错的地方互相印证，越看越像真的。** 本次就有一处这样的错误：多份文档一致地写着提交用的是前向校正损失， 而模型文件里记的是 SCE。是以产物为准，才把它纠过来的。

九、最终配置与产出

9.1 提交配置

项	取值
骨干	CLIP ViT-B/32（冻结，OpenAI 官方权重）
分类头	线性层 512 → 750，无隐层，含可学习 logit scale
损失	`--kind sce`，α = 0.1，β = 0.05
类平衡	`--cb cb`，β = 0.99
CL 保真度降权	开启，`CLIP_LO = 0.30`
前向校正	未启用
测试先验修正	`--prior none`
训练轮数	60（取验证最优轮，实测最优在第 45 轮附近）

9.2 产出

文件	说明
`pred_results.zip`	提交文件，内含 `pred_results.csv`，37,444 行
`class_map_used.csv`	本次使用的类别映射审计表
`head.pt`	分类头权重（约 1.5 MB）
`T.npy`	估计出的噪声转移矩阵（750×750）
`train_meta.json`	类别映射与元信息

提交文件经 `check_submission.py` 全项通过：行数正确、编号均为四位数字、 文件名与测试集完全一致、无重复、**预测覆盖 745/750 个类别**、 最大类占比 0.00676（无塌缩迹象）。

9.3 合规性

约束	状态	依据
必须使用 CLIP ViT-B/32	✅	`config.CLIP_HF_ID = openai/clip-vit-base-patch32`
仅限 OpenAI 官方公开权重	✅	加载的即上述官方权重
不得替换为其他视觉基础模型	✅	代码中无其他骨干加载路径
不得多模型集成 / 融合 / 投票	✅	单一模型、单一推理流程
不得使用商业闭源模型 API	✅	全程本地推理，无外部 API 调用
不得引入任何额外数据集	✅	只读本阶段 `data/train` 与 `data/test`
测试数据不得参与训练（含自监督）	✅	测试图片只做一次前向提特征，无梯度回传、无伪标签
自动清洗，不依赖人工清洗	✅	噪声处理全部由 CL 估计 + 损失加权自动完成

十、局限与后续方向

一、T 的估计没有解决。 两个估计器分别低估和反向高估，我们用"损失不依赖 T" 绕开了这个问题，但没有真正把它修好。少数类行不可辨识这一点是方法层面的限制。

二、容量假设的否证只覆盖 150 类。 全量是 750 类，任务更难， 冻结线在全量上未必同样饱和。如果后续要在全量上复测，取 100 类 × 100 张的短程对照 即可，成本约 1 小时。

三、诊断指标不能预测榜分。 第五章那些覆盖率、拟合率只用来排除有害配置， 真实水平必须以排行榜为准。

四、先验修正没有收益，但也没有害处。 如果后续确认测试集分布与训练集不同， 这个组件可以重新启用——它的实现与护栏都在，改一个参数即可。

五、当前方案的提升空间在特征侧而非分类头侧。 冻结特征决定了天花板； 在不违反规则的前提下，可考虑的只有损失与加权的进一步细调， 以及数据增强策略（本轮未系统做）。

> 本报告中所有标"实测"的数字都来自本工程的可复跑脚本， 对应的脚本名在各节内已注明。诊断类指

> 标的作用是排除有害配置与检测塌缩， **不能用于预测排行榜分数。**